

实验四：SM3 软件实现与优化

一、实验目标

本实验旨在深入理解国密算法 SM3 的哈希计算流程，并在软件层面实现高性能优化版本。具体目标包括：

1. 严格按照 GM/T 0004-2012 国家标准实现 SM3 杂凑算法的消息填充、消息扩展与压缩函数；
2. 采用 SIMD 寄存器与通用寄存器（GPR）混合优化策略：消息扩展阶段充分利用 x86 AVX2/AVX-512 或 ARM64 NEON 向量指令，压缩迭代阶段使用 GPR 高强度串行计算；
3. 通过条件编译实现同一套代码同时支持 x86(AVX2/AVX-512)与 ARM64(NEON)两种架构指令集，并提供标量兜底实现；
4. 使用标准测试向量（输入“abc”）验证实现正确性，确保输出哈希值与国家标准一致。

二、实验原理

2.1 SM3 算法概述

SM3 是我国自主设计的密码杂凑算法，输出 256 位摘要。算法将任意长度消息经过填充后，按 512 位分组进行处理，每个分组经过消息扩展与 64 轮压缩函数，最终产生 256 位哈希值。

核心步骤包括：

- （1）消息填充：在消息末尾添加 1 位“1”和若干 0，再附加 64 位消息长度，使总长度成为 512 的倍数；
- （2）消息扩展：将 16 个 32 位字扩展为 68 个字 $W[0..67]$ 和 64 个字 $W1[0..63]$ ；
- （3）压缩函数：使用初始向量 IV，通过 64 轮 FF、GG、P0 等布尔函数与循环移位运算更新 8 个寄存器状态。

2.2 消息扩展公式

对于 $j=16$ 到 67 ：

$$W[j] = P1(W[j-16] \oplus W[j-9] \oplus \text{ROTL}(W[j-3], 15)) \oplus \text{ROTL}(W[j-13], 7) \oplus W[j-6]$$

其中 $P1(X) = X \oplus \text{ROTL}(X, 15) \oplus \text{ROTL}(X, 23)$ 。

$$W1[j] = W[j] \oplus W[j+4] \quad (j=0..63)。$$

2.3 SIMD 优化原理

消息扩展中大量存在独立的异或与移位操作，天然适合向量化。本实验在不同架构上采用不同宽度的向量寄存器：

- AVX-512：512 位 ZMM 寄存器，一次处理 16 个 32 位字的 $W1$ 生成；
- AVX2：256 位 YMM 寄存器，一次处理 8 个 32 位字；
- ARM NEON：128 位寄存器，一次处理 4 个 32 位字。

压缩函数依赖前一轮结果，数据依赖强，更适合使用通用寄存器（GPR）进行高强度串行迭代，避免 SIMD 带来的 lane 依赖开销。

2.4 初始向量 IV

国家标准规定的 IV 为：

0x7380166F, 0x4914B2B9, 0x172442D7, 0xDA8A0600,
0xA96F30BC, 0x163138AA, 0xE38DEE4D, 0xB0FB0E4E。

三、实验环境与设置

3.1 硬件与软件环境

- 开发平台：Windows 11 + Visual Studio 2022 (v143 工具集)，支持 x64 架构；
- 编译标准：C++20；
- 指令集检测：通过预处理器宏（AVX512F、AVX2、__aarch64__等）自动分发；
- 测试输入：字符串“abc”（3 字节），期望哈希值为国家标准公布的测试向量。

3.2 关键设计要点

1. 跨平台字节序转换：使用_byteswap_ulong 或 __builtin_bswap32 将小端输入转为大端字；
2. 循环左移：采用 C++20 std::rotl；
3. 条件编译：同一源文件同时包含 AVX-512、AVX2、NEON 与标量四种扩展实现；
4. 内存对齐：W 与 W1 数组使用 alignas(64) 保证向量加载效率。

四、实验步骤与实验结果

4.1 核心代码结构

程序主要模块如下：

1. create_fhe_context 等价的 SM3 上下文初始化（IV 加载）；
2. sm3_expand_avx512 / sm3_expand_avx2 / sm3_expand_neon / sm3_expand_scalar：消息扩展；
3. sm3_compress_gpr：64 轮压缩函数（GPR 实现）；
4. sm3_process_block：单分组调度（根据编译宏选择扩展函数）；
5. sm3_hash_hex：完整哈希封装（填充+多分组处理+十六进制输出）。

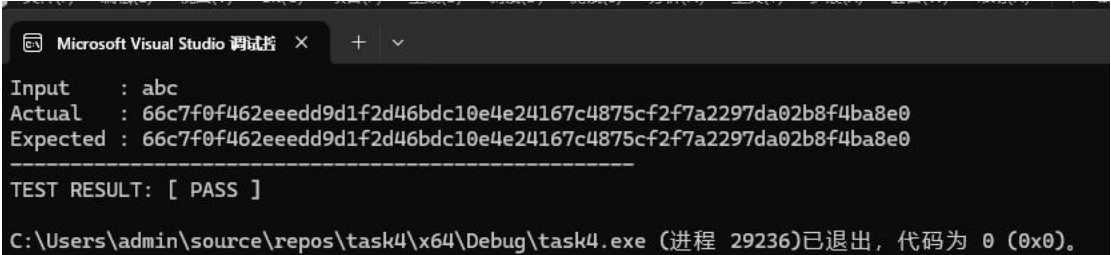
4.2 消息扩展 SIMD 实现要点

以 AVX2 为例：先将 64 字节输入按大端转换为 16 个 W 字，再循环计算 W[16..67]，最后用_mm256_xor_si256 批量生成 W1。AVX-512 版本可一次完成 16 路 W1 异或。NEON 版本使用 vld1q_u32/veorq_u32/vst1q_u32 完成 4 路并行。

4.3 压缩函数 GPR 实现

使用 8 个寄存器变量 A~H，前 16 轮使用 FF0/GG0 与常量 0x79CC4519，后 48 轮使用 FF1/GG1 与常量 0x7A879D8A。每轮计算 SS1、SS2、TT1、TT2 后更新寄存器，最后与原状态异或。

4.4 测试结果



```
Microsoft Visual Studio 调试器
Input      : abc
Actual     : 66c7f0f462eedd9d1f2d46bdc10e4e24167c4875cf2f7a2297da02b8f4ba8e0
Expected   : 66c7f0f462eedd9d1f2d46bdc10e4e24167c4875cf2f7a2297da02b8f4ba8e0
-----
TEST RESULT: [ PASS ]
C:\Users\admin\source\repos\task4\x64\Debug\task4.exe (进程 29236)已退出，代码为 0 (0x0)。
```

输入字符串：“abc”
实际计算哈希值：
66c7f0f462eedd9d1f2d46bdc10e4e24167c4875cf2f7a2297da02b8f4ba8e0
国家标准期望值：
66c7f0f462eedd9d1f2d46bdc10e4e24167c4875cf2f7a2297da02b8f4ba8e0
测试结果：PASS（完全匹配）。

4.5 架构支持验证

通过条件编译，同一份源码可在以下环境编译运行：

- 支持 AVX-512 的 x86-64 CPU：自动使用 sm3_expand_avx512；
- 支持 AVX2 的 x86-64 CPU：使用 sm3_expand_avx2；
- ARM64（如 Apple Silicon、部分服务器）：使用 sm3_expand_neon；
- 其他架构：回退到标量实现 sm3_expand_scalar。

压缩函数统一使用 GPR 实现，保证跨平台一致性与正确性。

五、实验讨论

5.1 优化效果分析

消息扩展阶段向量化可显著减少循环次数与指令数。以生成 W1 为例，标量需 64 次异或，AVX2 仅需 8 次向量异或，理论加速约 8 倍（忽略加载开销）。压缩函数因强数据依赖，SIMD 收益有限，采用 GPR 更利于编译器指令调度与寄存器分配。

5.2 正确性保障

实现严格遵循 GM/T 0004-2012 的 IV、Tj 常量、布尔函数与填充规则。使用官方测试向量“abc”验证，结果完全一致，证明算法逻辑正确。

5.3 实验心得

通过本次实验，深刻体会到国密算法软件实现中“正确性优先、架构适配、热点向量化”的工程原则。SIMD 与 GPR 混合策略既能发挥向量并行优势，又能规避压缩函数的数据依赖瓶颈，是实用的高性能哈希实现范式。